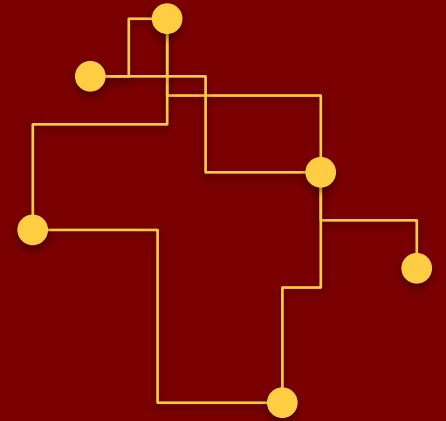


TEAM INTELLIAI2DOTO • PHASE 2

IntelliAI 2.0

The IBM MQ Migration Control Plane

Part 1 recommended the target topology. **Part 2 executes it — live, one app at a time, with zero message loss.**



16

MQ 9.4.5 QMs

7

apps migrated

12

state machine

0

messages lost

Migrating a live MQ estate is the hard part

Many apps share a few queue managers — one incident becomes everyone's incident. The fix is one QM per app. Doing it on a running estate is what's hard.

1 No message loss

Every in-flight message must be delivered — none stranded on the old queue, none duplicated.

2 No application change

Producers and consumers can't change connection strings or restart. They must not even notice.

3 No collateral impact

Migrating one app off a shared QM must not disturb the co-tenant apps still on it.

4 Full auditability

Every change recorded, causally ordered, and reversible — so it survives a real audit.

One control plane, one chokepoint to MQ

UI Control Plane

Next.js dashboard. Talks ONLY to the BCL. Holds no MQ credentials, issues no MQSC.



Business Control Layer (BCL)

FastAPI service — the sole point of contact with MQ + OpenShift. Deterministic engines, 3 advisory agents, Lamport-clocked audit log.

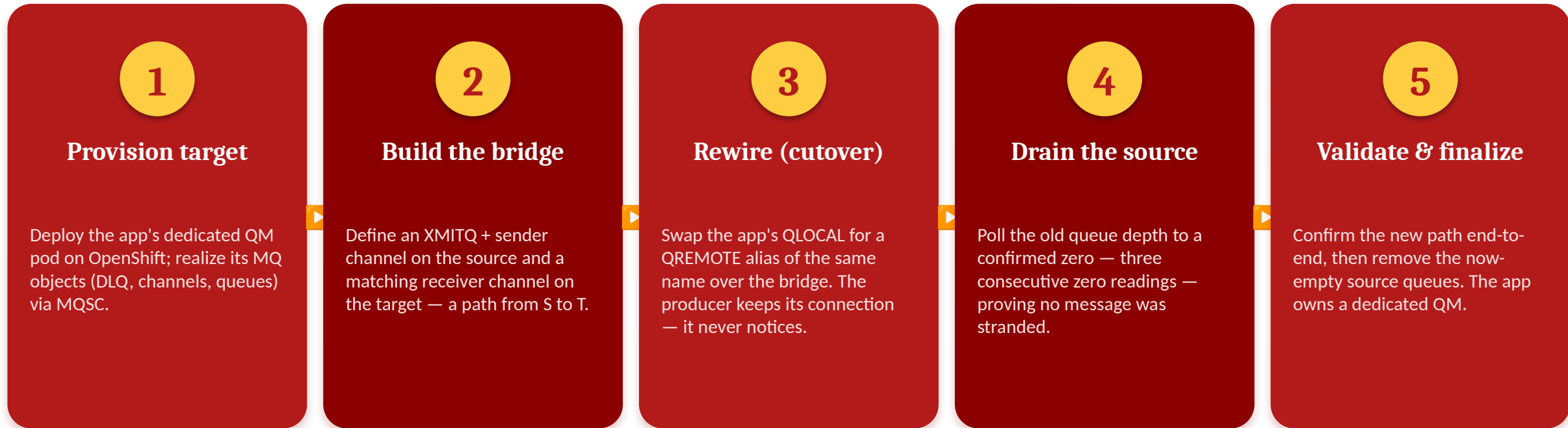


OpenShift · IBM MQ 9.4.5

16 real QM pods (9 shared source, 7 dedicated targets). BCL applies MQSC via oc exec.

The dependency direction is strict: the UI can't reach MQ — only the BCL can. One path means one place to validate, sequence, and record every single change.

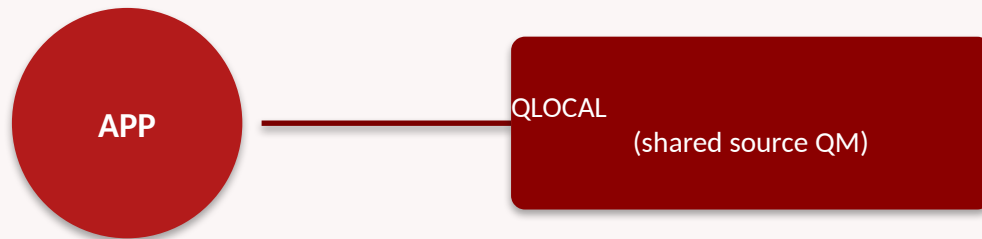
How one app migrates — five steps



The cutover is the clever part. Because we swap one queue for a same-named remote alias, MQ's own routing carries the messages to the new queue manager. The application keeps PUT-ting to the same name on the same connection — it never reconnects, and it never notices the move.

Transparent rewiring: the app never changes

Before — QLOCAL on shared QM



The app PUTs to a queue name on a connection string. That's all it knows.

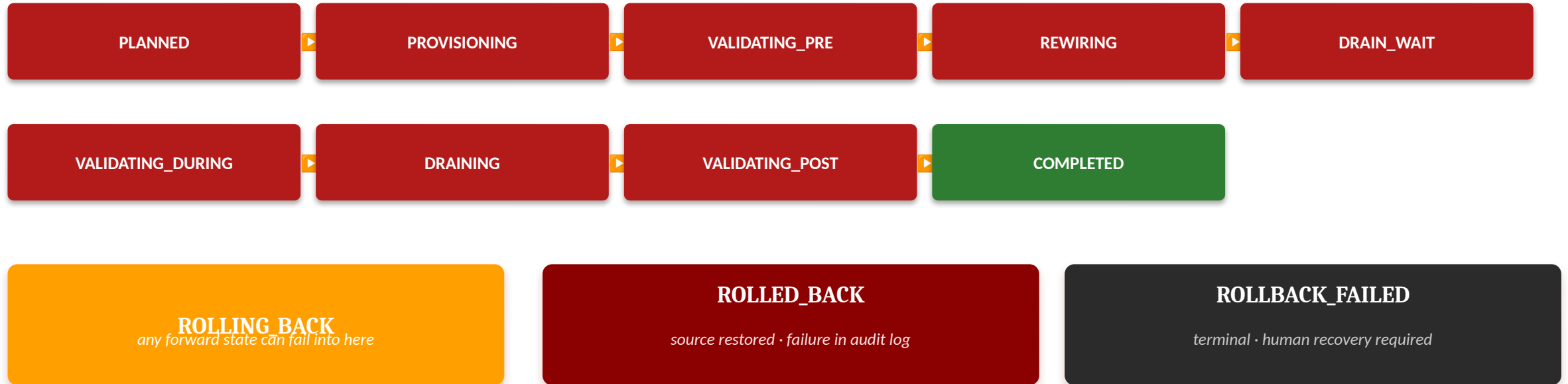
After — QREMOTE alias, same name



Same queue name, same connection. MQ routes over the bridge. Zero app changes.

The demo wow-moment: we show the producer pod's environment variables on screen — same DNS name, same port, before and after. Every message keeps flowing.

An explicit 12-state machine, not a script



Every app is always in a named state. A single assert_transition chokepoint makes an out-of-order transition impossible; the rules live in a pure, unit-testable module.

Every guarantee is checked, not asserted

✓ No message loss

Drain detection polls the source queue to a confirmed zero — three consecutive zero-depth readings, with no open handles — before the queue is removed.

✓ No app change

A remote-queue alias keeps the queue name and connection identical. The producer is never touched, never reconnects.

✓ No collateral impact

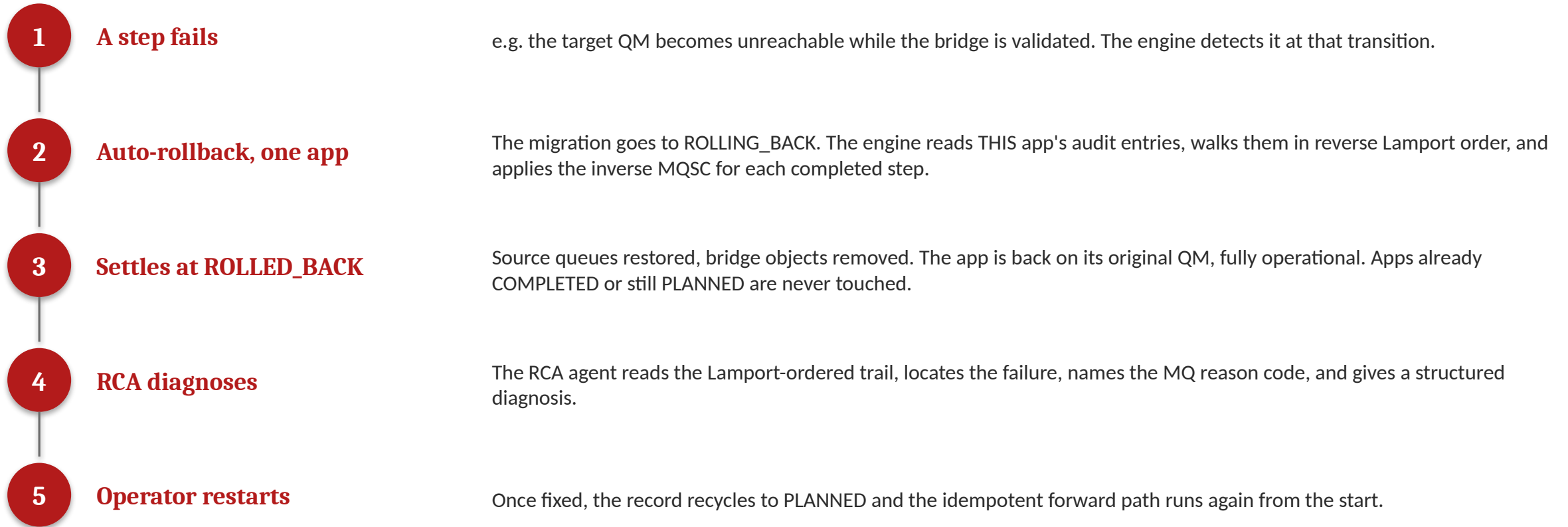
Blast-radius analysis enumerates every MQSC command the migration will issue and confirms ZERO touch a co-tenant's queues — the migration is gated on that count.

✓ Full auditability

Every state change writes a Lamport-clocked audit record. The audit log — not the UI, not the agents — is the system of record, exportable as evidence.

Per-app rollback in reverse Lamport order

The failure path is half the state machine. A failure is contained to one app — and only that app is reversed.



Three advisory agents — they inform, never act

1

Migration Planner

Produces a readable migration plan, the ordering rationale, the bridge/XMITQ names, and identified risks — for the operator to review before approving.

2

Operator Assistant

Answers free-text operational questions. Classifies intent, assembles read-only context from migrations and the audit log, and responds.

3

RCA Assistant

For a failed migration, reads the audit trail, locates the failure, names the MQ reason code, and produces a structured diagnosis with suggested checks.

Deterministic engines decide and act. Agents explain and advise. The audit log records.

All three run on the Tachyon gateway with Gemini 2.5 Pro. Every agent call is recorded in an ***agent_invocations*** table, with a per-invocation tool-call budget and a per-minute rate limit. An LLM is superb at planning and diagnosis — and unsuitable for issuing the MQSC that mutates a production queue manager. That split is the whole point.

Two models — each where it genuinely applies

Little's Law — drain prediction

$$L = \lambda W \rightarrow T_{\text{drain}} \approx L_0 / \mu$$

With the producer rewired away, no new messages arrive — so drain time is just the depth at cutover (L_0) over the consumer service rate (μ). It **predicts** the wait for the operator; the engine still **proves** the drain by polling depth to a confirmed zero.

Little (1961), Operations Research 9(3)

Absorbing Markov chain — reliability

$$N = (I - Q)^{-1}$$

The state machine IS an absorbing chain — COMPLETED, ROLLED_BACK, ROLLBACK_FAILED absorb. The fundamental matrix gives expected steps to absorption and the probability of each outcome. Shown **alongside** a separate empirical estimate from the real audit log — model and measurement, never conflated.

Kemeny & Snell (1960), Finite Markov Chains

Run end-to-end on a real MQ fleet

16

IBM MQ 9.4.5 queue managers

9 shared source + 7 dedicated targets, real pods on OpenShift roco-dev

7

applications migrated

Shared → dedicated, one at a time, each through its own 12-state machine

0

messages lost

Drain proven by polling depth to a confirmed zero on every cutover

100%

reversible

Every forward step has a clean inverse; per-app rollback verified live

Built like production, not a prototype



Audit as evidence

Lamport-clocked, append-only log with full request/response payloads and correlation IDs — one GET /audit/{lamport} answers who did what, when.



Evidence bundles

Per-migration zip: audit slice, MQSC commands, validation results, before/after topology snapshots — the kind of artifact that survives an internal audit.



Observability

Separate liveness/readiness endpoints, structured JSON logging, Prometheus metrics, OpenTelemetry tracing.



Idempotent & crash-safe

SQLite in WAL mode on a persistent volume; a restarted BCL recovers state and the forward path can be safely re-run.

What it doesn't claim — and what's next

Stated limits (earning trust)

- Formal next-state discipline, but no model checker is run — we don't claim machine-checked correctness.
- The Markov figure is a model, shown alongside — never merged with — the empirical audit estimate.
- Little's Law is a prediction; the drain is always confirmed by polling to zero.
- BCL runs single-instance; control-plane HA is out of scope, though state recovers on restart.
- Single OpenShift namespace; cross-app message dependencies out of scope.

Where it goes next

- Control-plane HA — multiple BCL instances behind a shared state store.
- Postgres at larger estate scale, replacing single-writer SQLite.
- Cross-namespace and cross-cluster migrations.
- Cross-application message dependencies (shared reply-to relationships).
- Model-checked verification of the state machine invariants.

NOW THE DEMO

Watch one app migrate — live

Plan → approve → provision → rewire → drain → validate. Then induce a failure, roll back, and read the RCA — all on real MQ.

Topology

16 QMs, 7 apps

Migrate

state machine + audit

Wow

app never reconnects

Rollback

chaos → reverse MQSC